Đồ án GK:

NHẬN DẠNG CHỮ SỐ VIẾT TAY MNIST

Bộ môn **Nhận dạng mẫu**

Khoa Toán-Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG TP.HCM

Giảng viên hướng dẫn: TS. Trương Phước Hưng

CN. Lê Phú Trường

Sinh viên thực hiện:

1. Vương Thành Đạt - MSSV: 24280058
2. Trần Nguyên Hưng - MSSV: 24280048
3. Phạm Phương Thảo - MSSV: 24280106

1 Tổng quan bài toán
1.1. Giới thiệu
1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
3.1. Lý thuyết MAP
3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
4.1. Lý thuyết
4.2. Cài đặt và thực nghiệm

5 MLP & CNN
5.1. Lý thuyết, Cài đặt thực nghiệm MLP
5.2. Lý thuyết, Cài đặt thực nghiệm CNN
5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
6.1. Lý thuyết MAP
6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
7.1. Khả năng phân loại
7.2. Thời gian huấn luyện
7.3. Độ trễ từng mẫu

8 Kết luận
8.1. Tổng kết
8.2. Hướng phát triển

Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

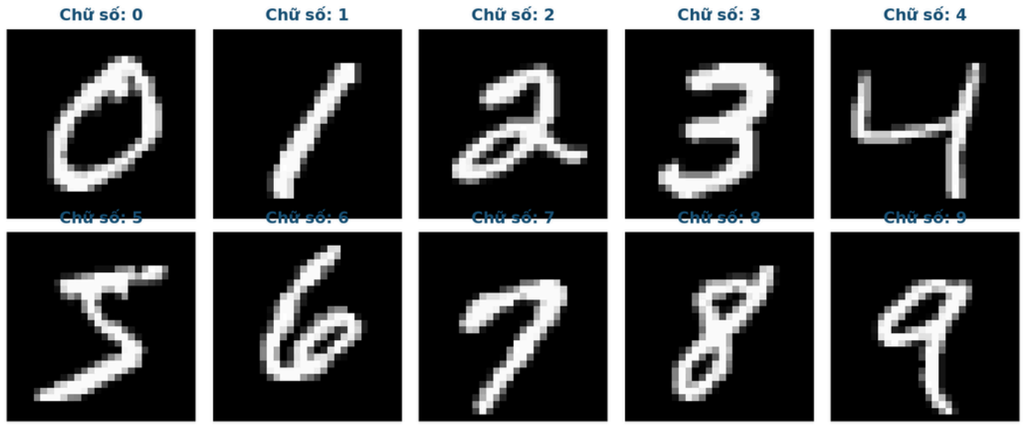
Giới thiệu

Bộ dữ liệu MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu kinh điển trong thị giác máy tính và nhận dạng mẫu:

- **Nguồn gốc:** Do Yann LeCun, Corinna Cortes và Christopher Burges tạo ra từ hai tập dữ liệu của Viện Tiêu chuẩn và Kỹ thuật Quốc gia Hoa Kỳ (NIST), gồm chữ số viết tay của học sinh trung học và nhân viên Cục Điều tra Dân số Hoa Kỳ.
- **Quy mô:** Gồm 70.000 ảnh chữ số viết tay, được chia thành 60.000 ảnh huấn luyện và 10.000 ảnh kiểm tra.
- **Định dạng ảnh:** Mỗi mẫu là một ảnh xám (grayscale) đơn kênh có kích thước 28 × 28 pixel, được căn giữa theo trọng tâm điểm ảnh.
- **Cường độ điểm ảnh:** Mỗi pixel có giá trị là số nguyên không âm nằm trong đoạn [0, 255]:
  - 0: Nền đen (tương ứng với không có mực).
  - 255: Nét chữ màu trắng (tương ứng với độ đậm mực tối đa).
- **Nhãn lớp:** Các số nguyên từ 0 đến 9



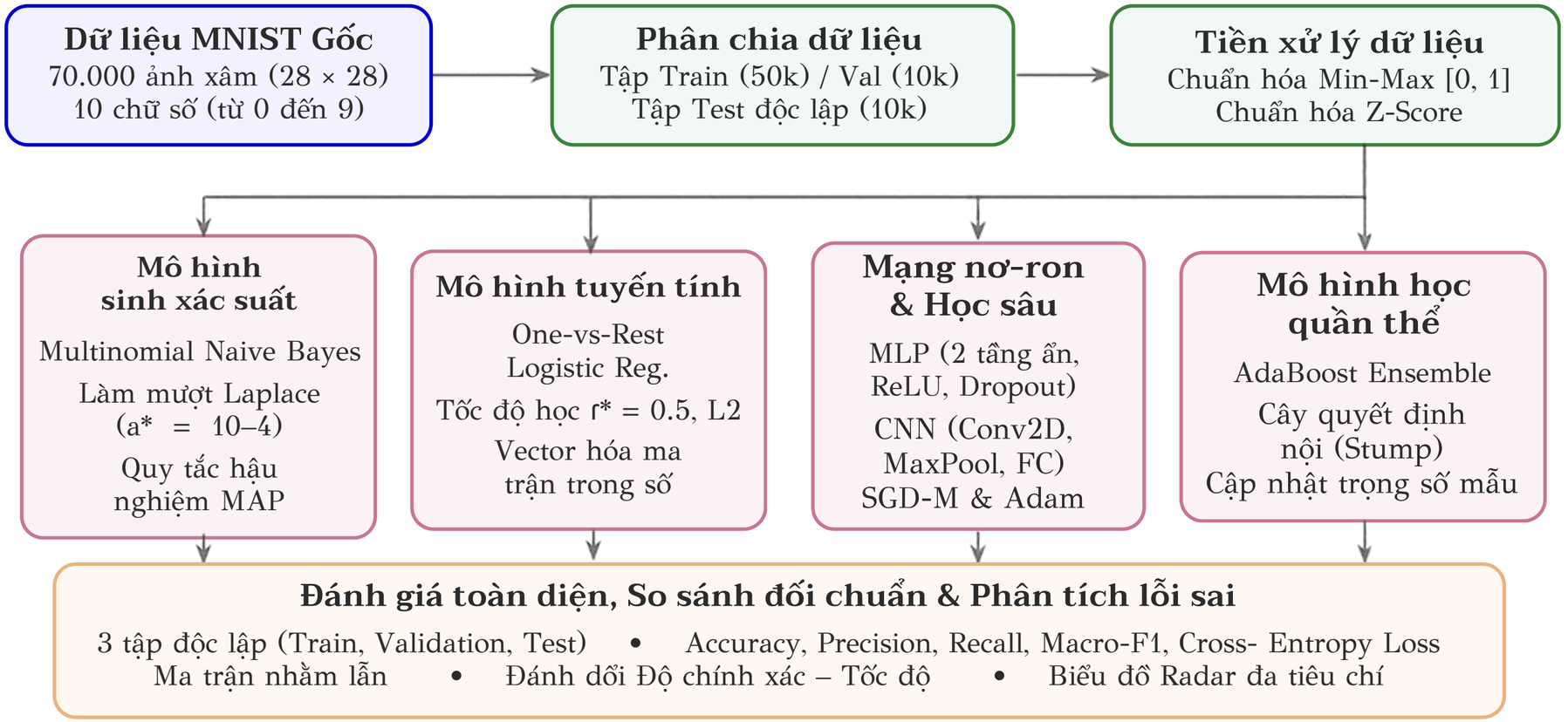
Viện Tiêu chuẩn và Kỹ thuật Quốc gia Hoa Kỳ (NIST)



Nhãn 10 chữ số từ 0 đến 9 của MNIST

Pipeline

Quy trình thực hiện đồ án được thực hiện đồng bộ, cụ thể thông qua 5 công đoạn được minh họa trong pipeline sau:



Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

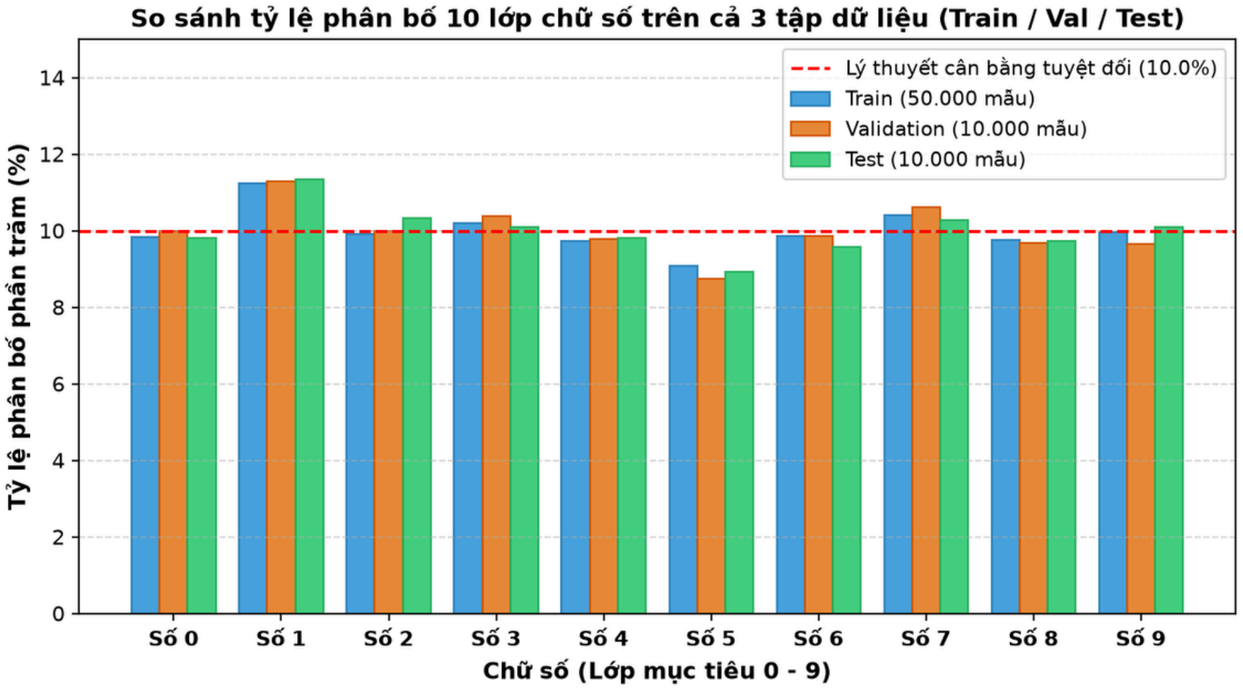
8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

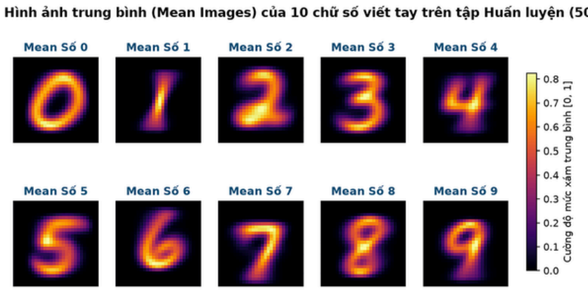
EDA & Preprocessing

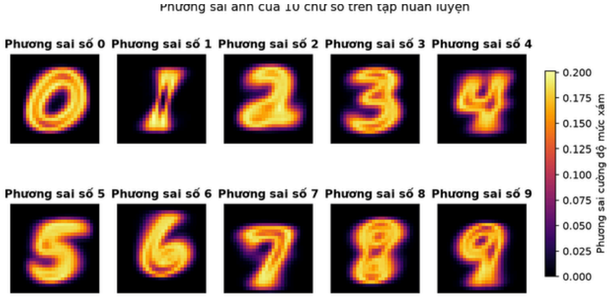
Qua quá trình **EDA**, nhóm nhận thấy:

- Dữ liệu hoàn toàn sạch, không chứa giá trị NaN hay mẫu trùng lặp.
- Các lớp phân bố đồng đều xoay quanh ngưỡng 10% (từ 8.92 đến 11.35)
- 67/784 pixel ở bốn mép viền luôn bằng 0 trên mọi mẫu (8.55% diện tích ảnh)

```
OUPUT KHI EDA:
    Số lượng missing value từng cột: 0
    Tổng số giá trị thiếu trong toàn bộ dataset: 0
    Số lượng dòng bị trùng lặp: 0
    Số đặc trưng có phương sai = 0: 67
    Số đặc trưng có giá trị âm: 0
    Số lượng nhãn không hợp lệ (y ∉ [0, 9]): 0
    Số lượng pixel không hợp lệ (X ∉ [0, 255]): 0
```







Preprocessing: Để đáp ứng đặc thù của từng nhóm thuật toán, nhóm đã xây dựng song song hai dạng biểu diễn dữ liệu đầu vào:

1. Chuẩn hóa Min-Max [0, 1]:

$$x_{norm} = \frac{x}{255.0}$$

2. Chuẩn hóa Z-Score:

$$x_{std} = \frac{x - \mu}{\sigma + \epsilon}$$

Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Lý thuyết MAP

Lý thuyết:

Mô hình Multinomial Naïve Bayes xem mỗi bức ảnh như một tập hợp các hạt mực phân bố trên không gian gồm **d = 784 pixel**. Dựa trên giả định các pixel độc lập có điều kiện theo từng lớp chữ số, nhãn dự đoán $\hat{y}$ được xác định thông qua quy tắc hậu nghiệm cực đại (Maximum A Posteriori - MAP) trên miền logarit:

$$\hat{y} = \arg\max_{c \in \{0,1,\ldots,9\}} \left[\ln P(y = c) + \sum_{j=1}^{d} x_j \ln \theta_{cj} \right]$$

Với θ_{cj} là xác suất hạt mực rơi vào pixel thứ j đối với chữ số c:

$$\hat{\theta}_{cj} = \frac{\mu_{cj} + \alpha}{\sum_{k=1}^{d} \mu_{ck} + d \cdot \alpha}$$

với μ_{cj} là cường độ trung bình của pixel j trong lớp c, và mẫu số được cộng thêm $d \cdot \alpha$ nhằm bảo đảm điều kiện chuẩn hóa xác suất:

$$\sum_{j=1}^{d} \hat{\theta}_{cj} = 1$$

Pipeline:



Lý thuyết MAP

Pseudocode:

Thuật toán 1 Huấn luyện và Dự đoán Multinomial Naïve Bayes

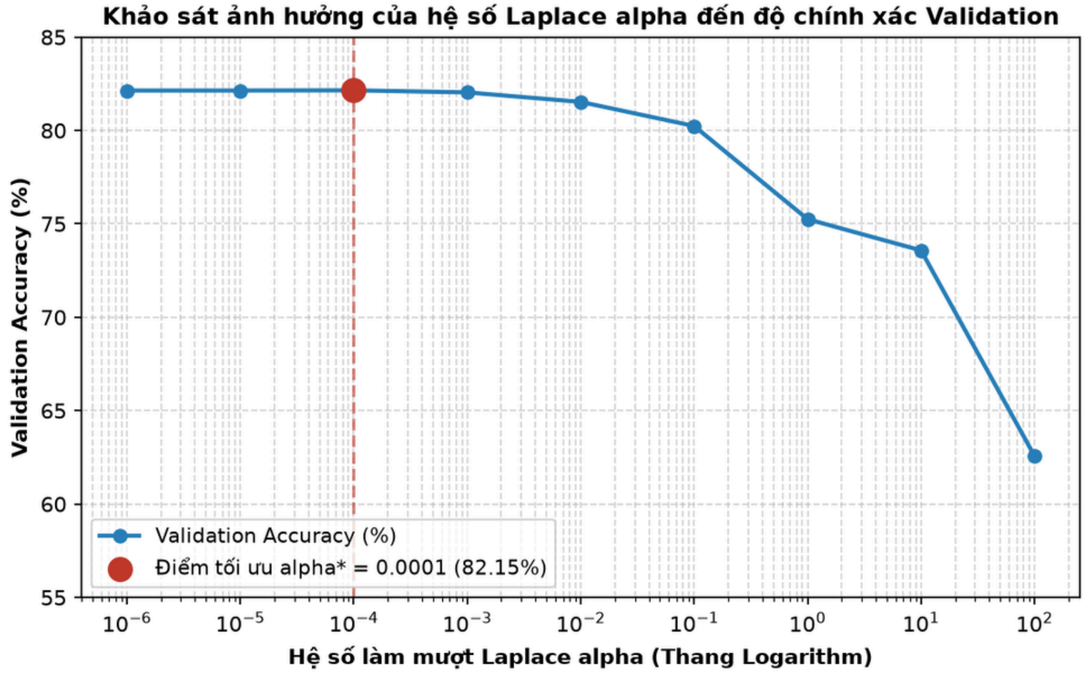
Đầu vào: Tập huấn luyện $\mathbf{X} \in [0,1]^{N \times d}$, nhãn $\mathbf{y} \in \{0,\ldots,9\}^N$, hệ số α, tập kiểm thử $\mathbf{X}_{\text{test}}$.
Đầu ra: Vector nhãn dự đoán $\hat{\mathbf{y}}_{\text{test}}$.

// Giai đoạn huấn luyện (Hàm fit):
for mỗi lớp $c \in \{0,1,\ldots,9\}$ **do**
    Lọc các mẫu thuộc lớp c: $\mathbf{X}_c \leftarrow \mathbf{X}[\mathbf{y} == c]$
    Xác suất tiên nghiệm: $\text{prior}[c] \leftarrow \frac{|\mathbf{X}_c|}{N}$
    Cường độ trung bình từng pixel: $\boldsymbol{\mu}_c \leftarrow \frac{1}{|\mathbf{X}_c|} \sum_{\mathbf{x} \in \mathbf{X}_c} \mathbf{x}$
end for
// Giai đoạn dự đoán (Hàm predict_log_proba & predict):
for mỗi lớp $c \in \{0,1,\ldots,9\}$ **do**
    Xác suất rơi mực Laplace: $\boldsymbol{\theta}_c \leftarrow \frac{\boldsymbol{\mu}_c + \alpha}{\sum_{j=1}^{d}(\mu_{cj} + \alpha)}$
    Log-likelihood của N_{test} mẫu: $\mathbf{L}_c \leftarrow \sum_{j=1}^{d} x_{ij} \ln \theta_{cj}$
    Điểm số hậu nghiệm: $\mathbf{S}[:,c] \leftarrow \mathbf{L}_c + \ln \text{prior}[c]$
end for
Gán nhãn có điểm lớn nhất: $\hat{\mathbf{y}}_{\text{test}} \leftarrow \arg\max_{c \in \{0,\ldots,9\}} \mathbf{S}[:,c]$
return $\hat{\mathbf{y}}_{\text{test}}$

Cài đặt & Thực nghiệm

Đánh giá α: Khảo sát lưới siêu tham số trên thang đo Logarit trên tập Validation:

$$\alpha \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 0.1, 1.0, 10.0, 100.0\}$$



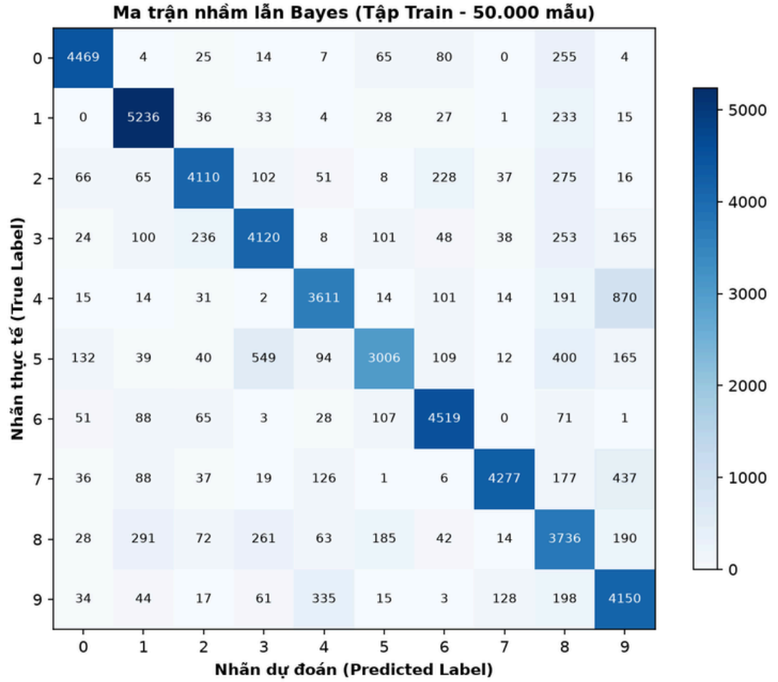
Kết quả thực nghiệm: Multinomial Na¨ıve Bayes ($\alpha\star = 10\text{^}{-}4$)

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	82.47	83.19	82.18	82.35	**0.06**
Validation	10.000	82.15	82.77	81.80	81.98	**0.03**
Test	10.000	**83.56**	84.24	83.25	83.41	**0.03**

Nhận xét:

Mô hình đạt tốc độ vượt trội (huấn luyện 50.000 mẫu trong 0.06 giây, suy luận 0.03 ms/mẫu).

Về khả năng nhận dạng, chữ số 0 (F1 = 92.77%) và 1 (F1= 92.26%) được phân loại tốt nhất nhờ nét chữ đặc trưng, ít chồng lấn; trong khi 5 có Recall thấp nhất (65.92%, bỏ sót 304 mẫu) và 8 có Precision thấp nhất (66.16%, nhầm lẫn với 398 mẫu khác).



Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Lý thuyết

Lý thuyết:

Logistic Regression là mô hình phân biệt tuyến tính tối ưu hóa trực tiếp xác suất phân lớp. Để giải quyết bài toán đa lớp (10 chữ số), nhóm áp dụng chiến lược One-vs-Rest (OvR), huấn luyện đồng thời 10 bộ phân loại nhị phân độc lập $c \in \{0, 1, \ldots, 9\}$

Hàm Sigmoid phân nhánh: Với mỗi mẫu ảnh được bổ sung bias $\tilde{\mathbf{x}} = [\mathbf{x}, 1]^T \in \mathbb{R}^{785}$ và vector trọng số $\mathbf{w}_c \in \mathbb{R}^{785}$, điểm số tuyến tính $z = \tilde{\mathbf{x}}\mathbf{w}_c$ được chuyển thành xác suất thông qua hàm Sigmoid phân nhánh nhằm chống tràn số:

$$\sigma(z) = \begin{cases} \frac{1}{1+e^{-z}}, & z \geq 0 \\ \frac{e^z}{1+e^z}, & z < 0 \end{cases}$$

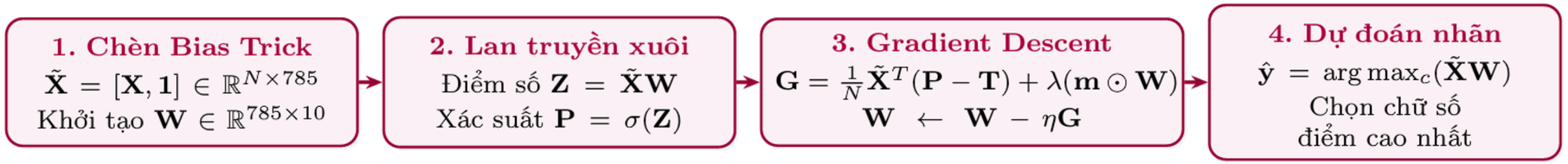
Hàm mất mát Binary Cross-Entropy với điều chuẩn L2:

$$J(\mathbf{w}_c) = \frac{1}{N}\sum_{i=1}^{N}[\ln(1 + e^{z_{ic}}) - t_{ic}z_{ic}] + \frac{\lambda}{2}\|\mathbf{m} \odot \mathbf{w}_c\|^2$$

Vector hóa ma trận: Thay vì lặp tuần tự, toàn bộ 10 mô hình được gộp thành ma trận trọng số $\mathbf{W} \in \mathbb{R}^{785\times10}$ và tối ưu hóa song song bằng Gradient Descent:

$$\nabla_{\mathbf{W}} J = \frac{1}{N}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{T}) + \lambda(\mathbf{m} \odot \mathbf{W}), \quad \mathbf{W} \leftarrow \mathbf{W} - \eta\nabla_{\mathbf{W}} J$$

Pipeline:



Lý thuyết

Pseudocode:

Thuật toán 2 Huấn luyện và Dự đoán One-vs-Rest Logistic Regression

Đầu vào: Tập huấn luyện $\mathbf{X} \in \mathbb{R}^{N \times d}$, nhãn $\mathbf{y} \in \{0, \ldots, 9\}^N$, tốc độ học η, hệ số λ, số epoch `max_epochs`, tập kiểm thử $\mathbf{X}_{\text{test}}$.

Đầu ra: Vector nhãn dự đoán $\hat{\mathbf{y}}_{\text{test}}$.

// Giai đoạn huấn luyện (Hàm fit):

Thêm cột bias: $\tilde{\mathbf{X}} \leftarrow [\mathbf{X}, \mathbf{1}] \in \mathbb{R}^{N \times (d+1)}$

Tạo ma trận nhãn One-hot: $\mathbf{T} \in \{0, 1\}^{N \times 10}$ với $\mathbf{T}[:, c] \leftarrow (\mathbf{y} == c)$

Khởi tạo ma trận trọng số $\mathbf{W} \leftarrow \mathbf{0} \in \mathbb{R}^{(d+1) \times 10}$, mặt nạ không phạt bias $\mathbf{m} = [1, \ldots, 1, 0]^T$

for epoch $= 1 \rightarrow$ max_epochs **do**

    Ma trận điểm số: $\mathbf{Z} \leftarrow \tilde{\mathbf{X}}\mathbf{W}$

    Ma trận xác suất: $\mathbf{P} \leftarrow \sigma(\mathbf{Z})$

    Ma trận Gradient: $\mathbf{G} \leftarrow \frac{1}{N}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{T}) + \lambda(\mathbf{m} \odot \mathbf{W})$

    Cập nhật song song 10 bộ trọng số: $\mathbf{W} \leftarrow \mathbf{W} - \eta\mathbf{G}$

end for

// Giai đoạn dự đoán (Hàm predict):

Thêm bias tập kiểm thử: $\tilde{\mathbf{X}}_{\text{test}} \leftarrow [\mathbf{X}_{\text{test}}, \mathbf{1}]$

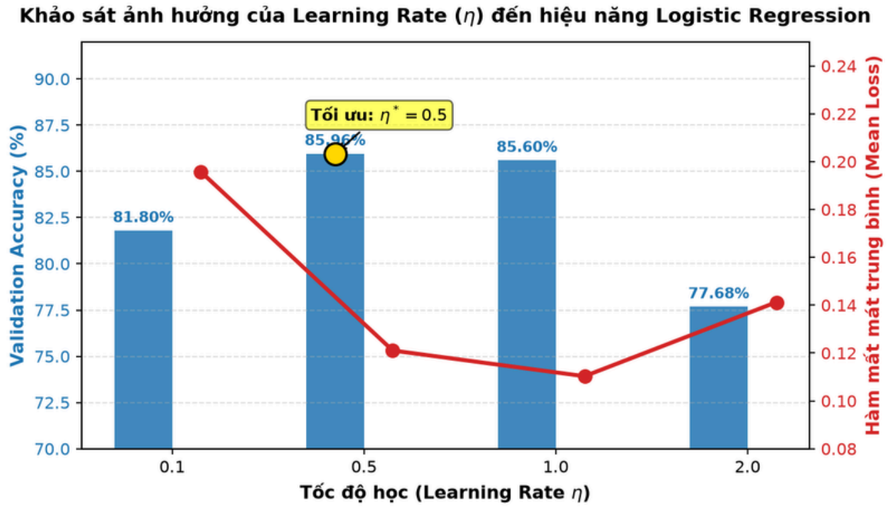
Tính ma trận điểm số: $\mathbf{Z}_{\text{test}} \leftarrow \tilde{\mathbf{X}}_{\text{test}}\mathbf{W}$

Gán nhãn có điểm cao nhất: $\hat{\mathbf{y}}_{\text{test}} \leftarrow \arg\max_{c \in \{0,\ldots,9\}} \mathbf{Z}_{\text{test}}[:, c]$

return $\hat{\mathbf{y}}_{\text{test}}$

Cài đặt & Thực nghiệm

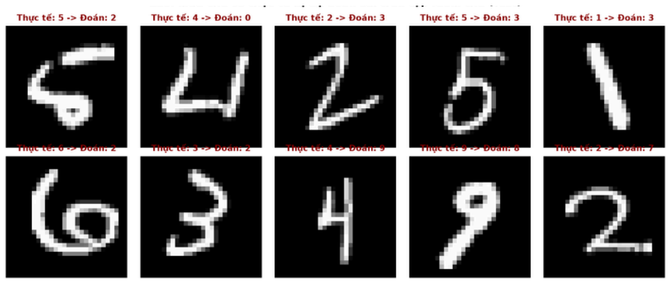
Khảo sát tốc độ học (η): Quá trình quét dải giá trị
$\eta \in \{0.1, 0.5, 1.0, 2.0\}$
trên tập Validation, xác định điểm cực trị tại $\eta^* = 0.5$



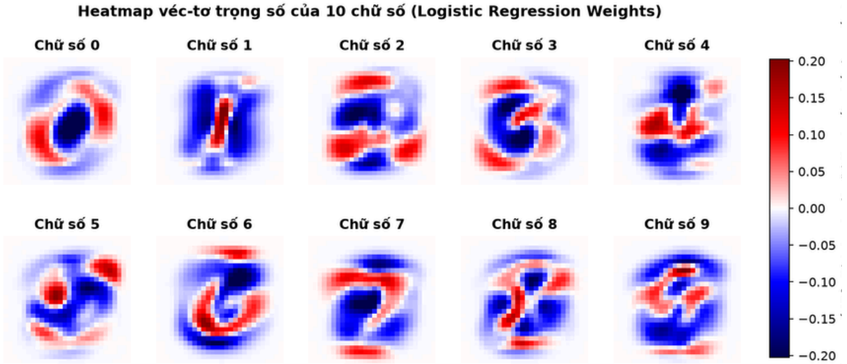
Kết quả thực nghiệm: One-vs-Rest Logistic Regression ($\eta* = 0.5$, 30 epochs)

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	86.00	86.04	85.72	85.71	10.70
Validation	10.000	85.96	85.96	85.64	85.69	**0.03**
Test	10.000	**86.84**	86.87	86.58	86.55	**0.03**

Ảnh dự đoán sai: Nhận diện tốt số 0 và 1 nhưng gặp khó khăn ở số 5 và 8. Các lỗi nhầm lẫn chủ yếu (4 - 9, 5 - 3) xuất phát từ ranh giới tuyến tính không thể mô hình hóa nét cong phức tạp,



Heatmap trọng số:
_Cộng điểm vùng wj > 0, trừ điểm vùng wj < 0
_Kết hợp điều chuẩn L2



Nhận xét:

Mô hình đạt độ chính xác 86.84% với tốc độ vượt trội (0.0029 ms/mẫu)

Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Lý thuyết MLP

Lý thuyết:

Dữ liệu được chuẩn hóa Z-score từ phần Preprocessing được thực hiện lan truyền thuận qua các lớp với ReLU, sau đó dùng Softmax để dự đoán.

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)}), \qquad p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

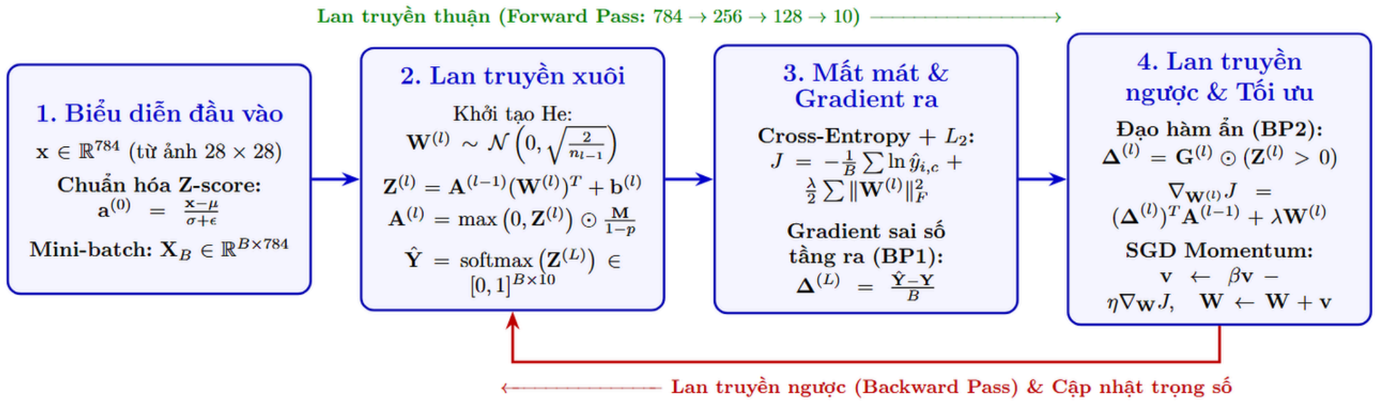
Mô hình được tối ưu bằng hàm mất mát Cross-Entropy kết hợp điều chuẩn L2; gradient được tính bằng lan truyền ngược và cập nhật trọng số với SGD Momentum:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\log p_{i,y_i} + \frac{\lambda}{2}\sum_{l}|\mathbf{W}^{(l)}|2^2, \qquad \mathbf{v}t = \beta\mathbf{v}t - 1 + \nabla\boldsymbol{\theta}\mathcal{L}, \qquad \boldsymbol{\theta}t = \boldsymbol{\theta}t - 1 - \eta\mathbf{v}_t$$

Thuật toán lan truyền ngược:

$$\boldsymbol{\delta}^{(L)} = \frac{1}{N}(\mathbf{P} - \mathbf{Y}) \qquad \boldsymbol{\delta}^{(l)} = \left(\boldsymbol{\delta}^{(l+1)}\mathbf{W}^{(l+1)}\right) \odot f'(\mathbf{Z}^{(l)}), \qquad \frac{\partial J}{\partial \mathbf{W}^{(l)}} = (\boldsymbol{\delta}^{(l)})^{\top}\mathbf{A}^{(l-1)}$$

Pipeline:



Lý thuyết MLP

Pseudocode:

Thuật toán 3 Huấn luyện Mạng nơ-ron nhiều tầng (MLP Backpropagation with Momentum)

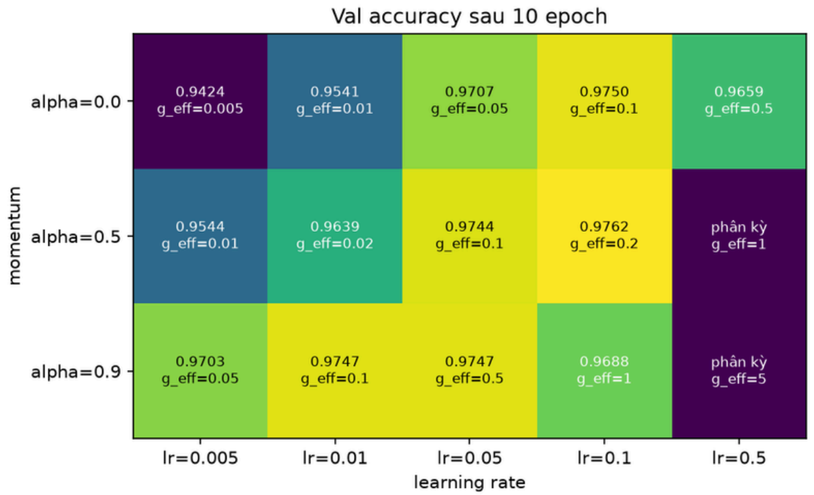
Đầu vào: Dữ liệu $\mathbf{X}$, nhãn $\mathbf{y}$, số tầng L, tốc độ học η, hệ số quán tính β, batch size B.
Đầu ra: Tập trọng số và độ lệch tối ưu $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$.

Khởi tạo trọng số He $\mathbf{W}^{(l)} \sim \mathcal{N}(0, \sqrt{2/n_{l-1}})$, bias $\mathbf{b}^{(l)} \leftarrow \mathbf{0}$, vận tốc $\mathbf{v}^{(l)} \leftarrow \mathbf{0}$.
for mỗi mini-batch $(\mathbf{X}_B, \mathbf{Y}_B)$ **do**
    Forward: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)})$; tầng cuối Softmax $\mathbf{P} = \mathrm{softmax}(\mathbf{z}^{(L)})$.
    Backward: Lỗi tầng ra $\boldsymbol{\delta}^{(L)} = \mathbf{P} - \mathbf{T}$; lỗi tầng ẩn $\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}] \odot \mathbf{1}(\mathbf{z}^{(l)} > 0)$.
    Update: $\mathbf{v}^{(l)} \leftarrow \beta\mathbf{v}^{(l)} + \eta\left(\frac{1}{B}\boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T + \lambda\mathbf{W}^{(l)}\right)$, $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \mathbf{v}^{(l)}$.
end for
return $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$

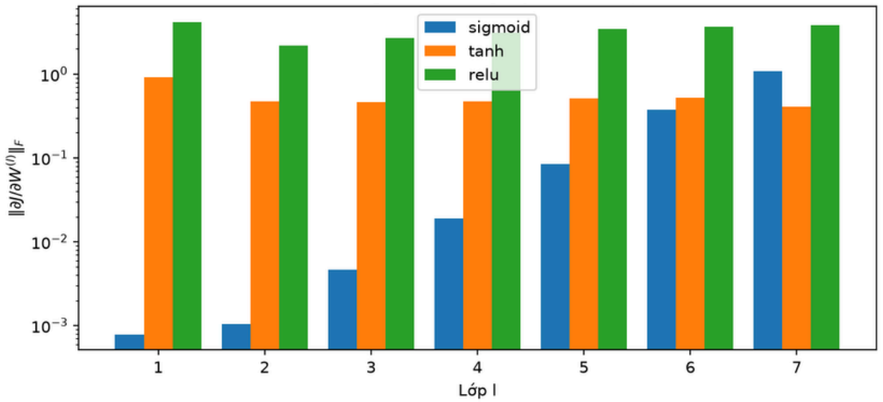
Cài đặt & Thực nghiệm

Khảo sát ảnh hưởng của learning rate và momentum:

$\gamma \in [0.005, 0.01, 0.05, 0.1, 0.5]$ $\alpha \in [0, 0.5, 0.9]$



Khảo sát các Acivation function bằng Vanishing Gradient:



Khảo sát ảnh hưởng của kích thước batch size:

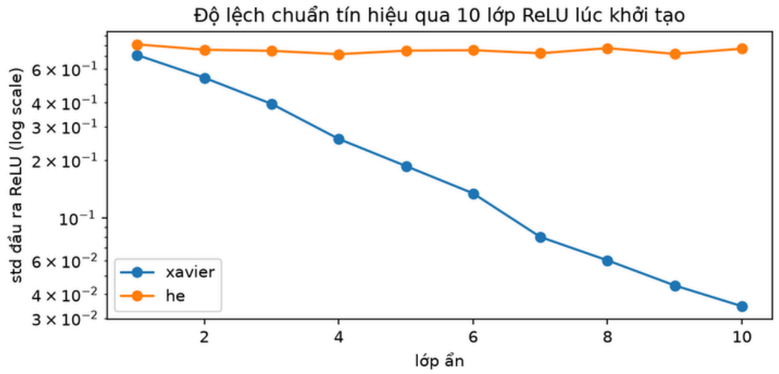
Cho batch_size (16, 64, 256, 1024) đến hiệu năng MLP trong 10 epoch, với $\gamma = 0.1$ và $\alpha = 0.5$

batch size	số tính toán	thời gian	val acc
16	15625	3.97	0.9611
64	3910	1.57	**0.9794**
256	980	0.72	0.9737
1024	245	0.49	0.9660

Khảo sát Độ rộng và Độ sâu của Mạng

kiến trúc	tham số	train acc	val acc
784 -> 64 -> 10	50,890	0.9937	0.9712
784 -> 256 -> 10	**203,530**	1.0000	**0.9763**
784 -> 1024 -> 10	814,090	1.0000	0.9790
784 -> 256 -> 128 -> 10	235,146	1.0000	0.9794
784 -> 256 -> 256 -> 256 -> 256 -> 10	400,906	0.9938	0.9730

Khảo sát khởi tạo bộ trọng với ReLU:



Lý thuyết CNN

Lý thuyết:

Dùng data đã chuẩn hóa về [0;1], CNN dùng các bộ lọc tích chập để nhận diện đặc trưng như nét và cạnh; sau đó ReLU và Max-Pooling giúp giữ đặc trưng nổi bật, giảm kích thước dữ liệu. Cuối cùng, các đặc trưng được đưa qua lớp Fully connected:

Ảnh 28 × 28 → Convolution (32 bộ lọc 4 × 4) → ReLU → Max-Pooling 2 × 2 → Dense 128 → 10 lớp

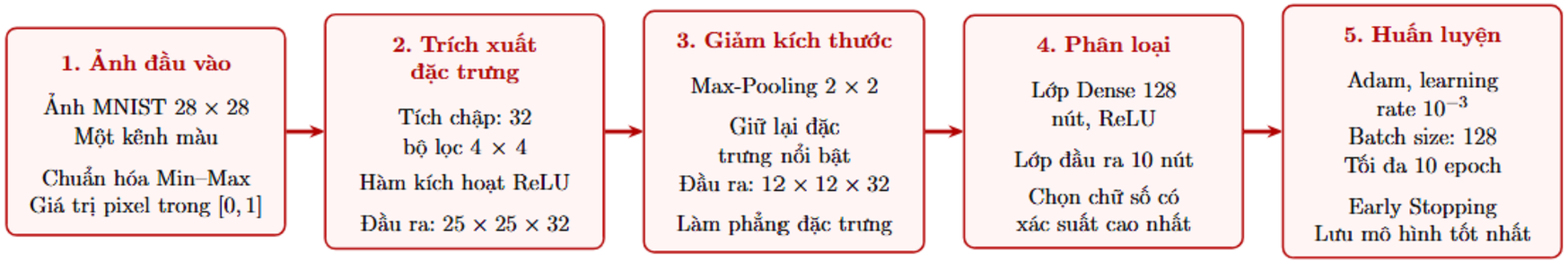
Phép tích chập tính điểm đặc trưng tại mỗi vị trí: $z_{k,i,j} = \sum_{c,u,v} K_{k,c,u,v}\, x_{c,i+u,j+v} + b_k, \qquad a_{k,i,j} = \mathrm{ReLU}(z_{k,i,j})$

Max-Pooling lấy giá trị lớn nhất trong mỗi vùng 2 × 2: $p_{k,i,j} = \max_{(u,v)\in\{0,1\}^2} a_{k,\,2i+u,\,2j+v}$

Sau khi làm phẳng đặc trưng, Softmax chuyển đầu ra thành xác suất của từng chữ số:

$$P(y=k \mid x) = \frac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}, \qquad \hat{y} = \arg\max_k P(y=k \mid x)$$

Pipeline:



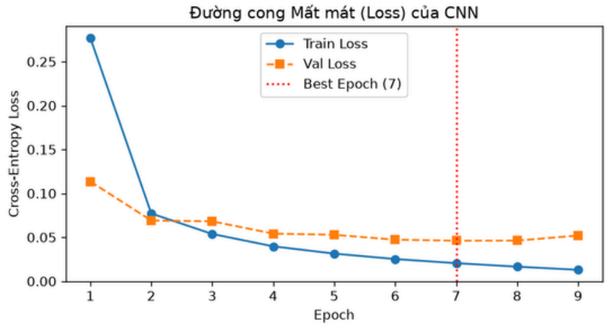
Cài đặt & Thực nghiệm

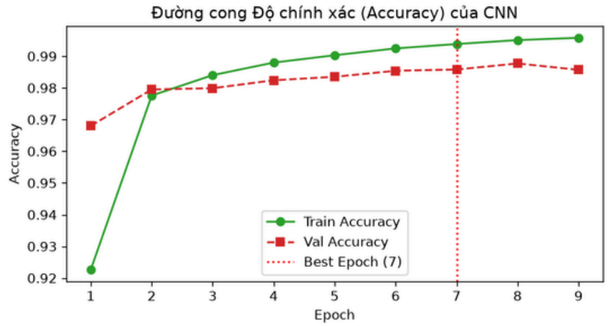
Huấn luyện CNN với Adam và Early Stopping:

Với learning rate (10^−3)), batch size (128), tối đa 10 epoch và patience (2):

```
Epoch   1 | 6.4s | Train loss 0.2771, acc 0.9226 | Val loss 0.1135, acc 0.9680
Epoch   2 | 5.6s | Train loss 0.0771, acc 0.9776 | Val loss 0.0691, acc 0.9795
Epoch   3 | 5.6s | Train loss 0.0538, acc 0.9840 | Val loss 0.0681, acc 0.9799
Epoch   4 | 5.9s | Train loss 0.0397, acc 0.9880 | Val loss 0.0542, acc 0.9824
Epoch   5 | 5.8s | Train loss 0.0314, acc 0.9903 | Val loss 0.0528, acc 0.9835
Epoch   6 | 5.6s | Train loss 0.0252, acc 0.9925 | Val loss 0.0473, acc 0.9854
Epoch   7 | 5.6s | Train loss 0.0205, acc 0.9938 | Val loss 0.0461, acc 0.9858
Epoch   8 | 5.7s | Train loss 0.0165, acc 0.9951 | Val loss 0.0462, acc 0.9877
Epoch   9 | 5.4s | Train loss 0.0130, acc 0.9958 | Val loss 0.0520, acc 0.9857
Early Stopping: val_loss không cải thiện trong 2 epoch liên tiếp.
Huấn luyện xong trong 51.61s | Epoch tốt nhất: 7 | Val loss tốt nhất: 0.0461
```

Kết quả huấn luyện và đánh giá mô hình CNN





Nhận xét, so sánh MLP và CNN:

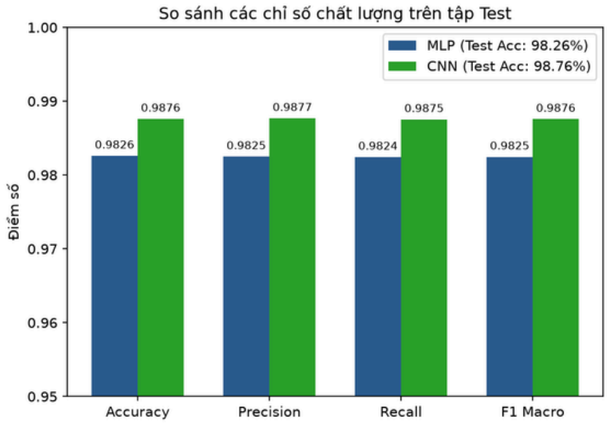
Bảng 1: Kết quả thực nghiệm của MLP.

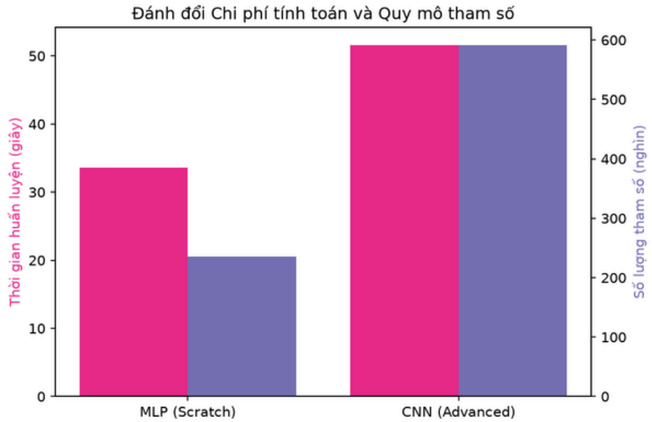
Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Train (s)	Predict (s)
Train	50.000	100.00	100.00	100.00	100.00	33.5715	0.1142
Validation	10.000	97.88	97.85	97.87	97.86	33.5715	**0.0230**
Test	10.000	**98.26**	98.25	98.24	98.25	33.5715	**0.0216**

Bảng 2: Kết quả thực nghiệm của CNN.

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Train (s)	Predict (s)
Train	50.000	99.65	99.65	99.65	99.65	51.6119	2.1970
Validation	10.000	98.58	98.57	98.57	98.57	51.6119	0.4391
Test	10.000	**98.76**	98.77	98.75	98.76	51.6119	**0.4285**

Trực quan các chỉ số chất lượng và chi phí tính toán





Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Lý thuyết MAP

Lý thuyết:

AdaBoost gộp nhiều weak learner thành một mô hình mạnh. Khác với Bagging (các mô hình độc lập, song song), AdaBoost huấn luyện tuần tự: mô hình sau tập trung sửa lỗi mà mô hình trước mắc phải bằng cách tăng trọng số cho các mẫu bị phân loại sai.

Cho tập huấn luyện gồm n mẫu. Ban đầu, trọng số của các mẫu được phân bổ đều: $w_1(i) = \frac{1}{n}, \quad \forall i = 1, \ldots, n$

tại mỗi vòng lặp t = 1, 2,..., T, thuật toán thực hiện tuần tự 4 bước:

B1: Huấn luyện mô hình yếu (h_t):

$$h_t = \arg\min_h \sum_{i=1}^{n} w_t(i) \cdot \mathbb{I}(h(x_i) \neq y_i)$$

B2: Tính tỷ lệ lỗi có trọng số (ε_t):

$$\varepsilon_t = \sum_{i=1}^{n} w_t(i) \cdot \mathbb{I}(h_t(x_i) \neq y_i)$$

B3: Xác định trọng số đóng góp của mô hình (alpha_t):

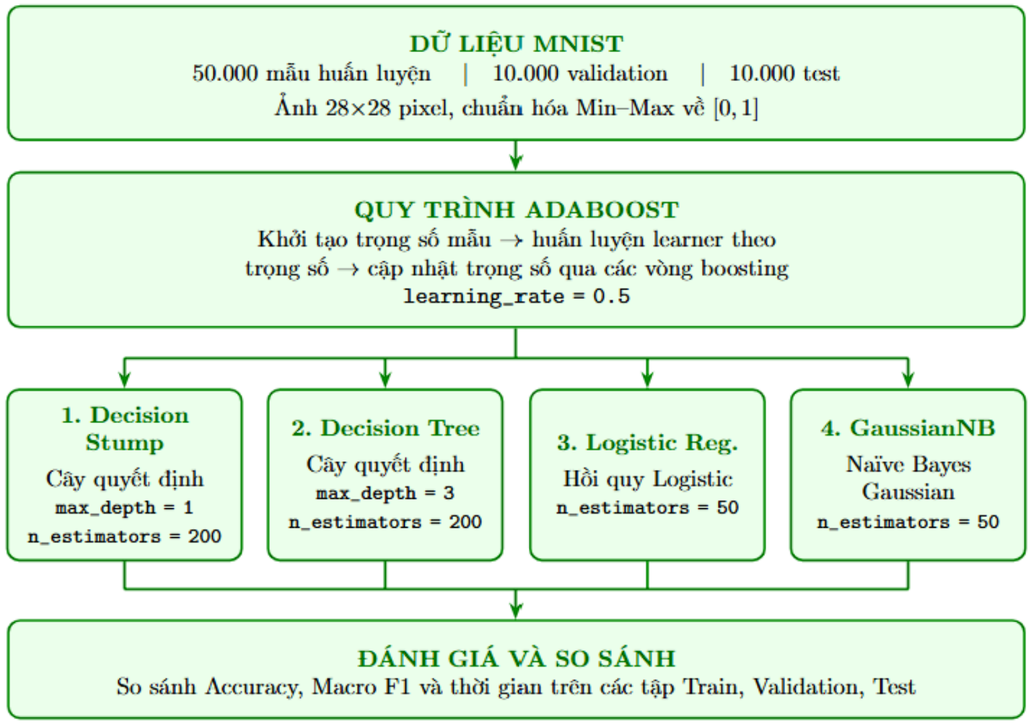
$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$$

B4: Cập nhật và chuẩn hóa trọng số dữ liệu (w_{t+1}):

Tăng trọng số các mẫu dự đoán sai, giảm trọng số dự đoán đúng:

$$w_{t+1}(i) = \frac{w_t(i) \cdot \exp\left(-\alpha_t y_i h_t(x_i)\right)}{Z_t} \qquad H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$$

Pipeline:



Lý thuyết MAP

Pseudocode:

Thuật toán 4 Huấn luyện Học quần thể AdaBoost đa lớp (SAMME)

Đầu vào: Tập dữ liệu $\{(x_i, y_i)\}_{i=1}^{N}$, số lớp K, số vòng T, tốc độ học η.

Đầu ra: Bộ phân loại tổng hợp $H(x) = \arg\max_{k} \sum_{t=1}^{T'} \alpha_t \, \mathbf{1}[h_t(x) = k]$.

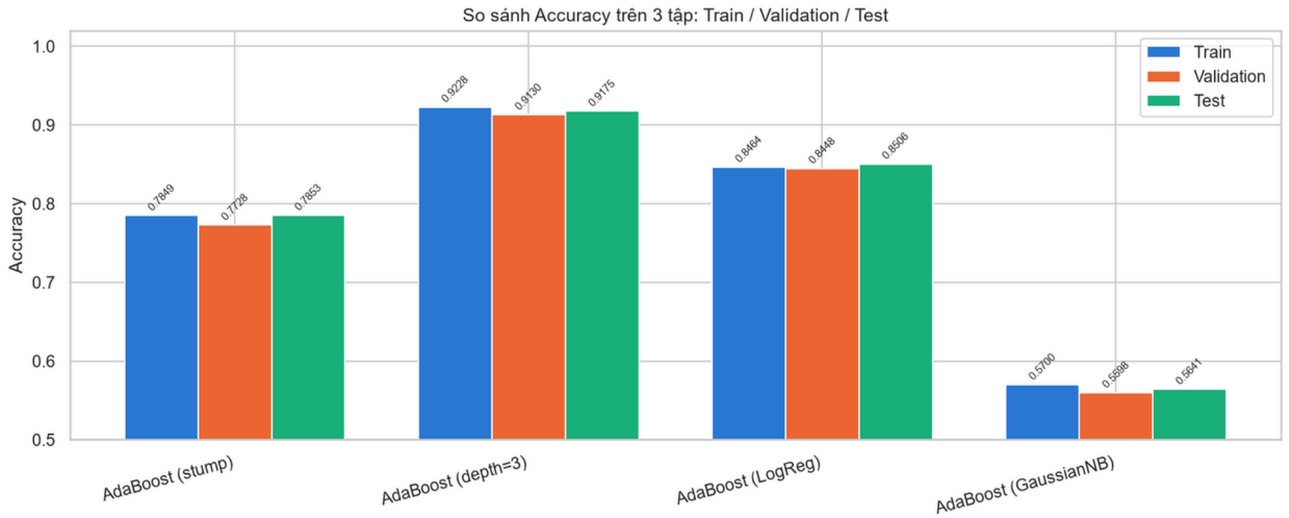
Khởi tạo trọng số mẫu phân bố đều: $w_1(i) = 1/N, \ \forall i \in \{1, \ldots, N\}$.
for $t = 1 \to T$ **do**
    Huấn luyện h_t: $h_t = \arg\min_{h \in \mathcal{H}} \sum_{i=1}^{N} w_t(i) \, \mathbf{1}[h(x_i) \neq y_i]$.
    Tính sai số có trọng số: $\varepsilon_t = \sum_{i=1}^{N} w_t(i) \, \mathbf{1}[h_t(x_i) \neq y_i]$.
    if $\varepsilon_t = 0$ **then return** $h_t(x)$ (dừng sớm).
    if $\varepsilon_t \geq 1 - 1/K$ **then** loại h_t và dừng boosting.
    Trọng số mô hình: $\alpha_t = \eta \left[\ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right) + \ln(K-1) \right]$.
    Cập nhật trọng số mẫu: $\tilde{w}_{t+1}(i) = w_t(i) \exp(\alpha_t \, \mathbf{1}[h_t(x_i) \neq y_i])$.
    Chuẩn hóa: $Z_t = \sum_j \tilde{w}_{t+1}(j), \quad w_{t+1}(i) = \frac{\tilde{w}_{t+1}(i)}{Z_t}$.
end for
return $H(x) = \arg\max_{k \in \{0,\ldots,K-1\}} \sum_{t=1}^{T'} \alpha_t \, \mathbf{1}[h_t(x) = k]$.
    Trong đó $T' \leq T$ là số vòng thực tế đã huấn luyện.

Cài đặt & Thực nghiệm

Huấn luyện và đánh giá các mô hình:

Mô hình	Accuracy	Macro F1	Precision	Recall	Train (s)	Predict (s)
AdaBoost (depth=3)	**0.9175**	**0.9170**	**0.9181**	**0.9170**	11019.2396	2.6681
AdaBoost (LogReg)	0.8506	0.8485	0.8511	0.8479	48.3919	**0.3013**
AdaBoost (stump)	0.7853	0.7837	0.7880	0.7826	241.9720	2.4197
AdaBoost (GaussianNB)	0.5641	0.5083	0.7158	0.5563	**46.7646**	7.8411

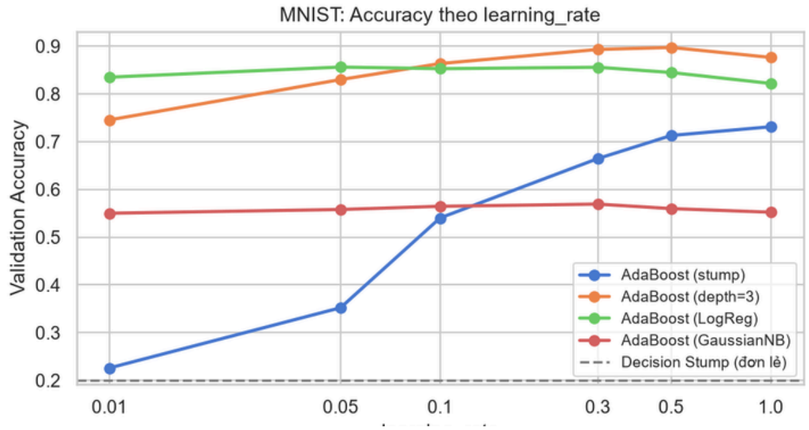
So sánh Accuracy trên 3 tập (Train/Val/Test)

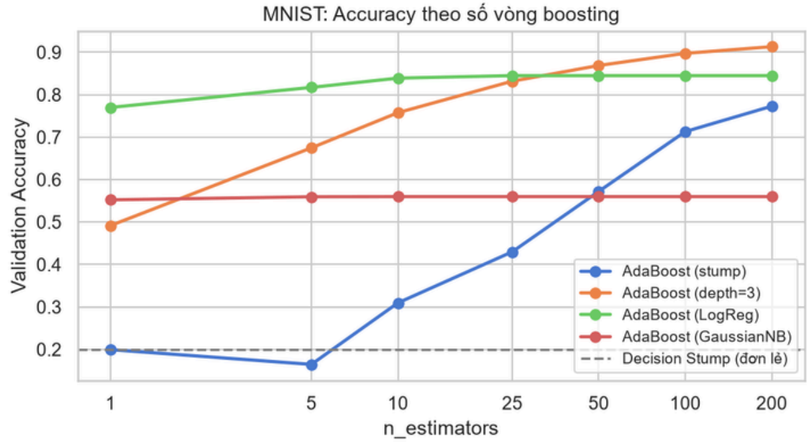


Nhận xét:

AdaBoost (depth=3) dẫn đầu trên tập Test với 91.62% Accuracy. Việc lựa chọn mô hình yếu đóng vai trò quyết định hơn tinh chỉnh siêu tham số, tạo ra mức chênh lệch hiệu năng lên tới 37%.

Khảo sát ảnh hưởng của learning rate và số vòng boosting:





Hiệu năng AdaBoost phụ thuộc vào sự cân bằng giữa độ mạnh của weak learner, learning_rate và n_estimators: cây độ sâu 3 thường đạt độ chính xác cao, còn tăng số vòng boosting cải thiện kết quả nhưng làm tăng thời gian huấn luyện.

Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

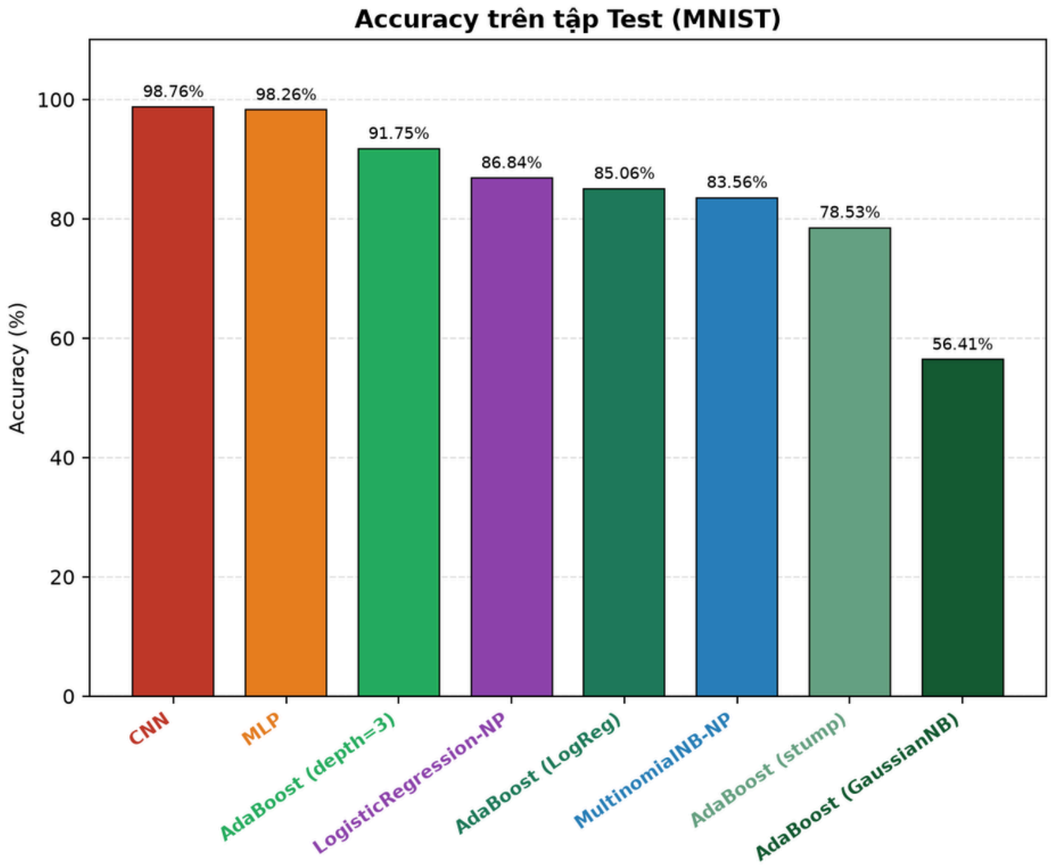
7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Khả năng phân loại

Bảng tổng hợp metrics và thời gian:

Mô hình	Accuracy	Precision	Recall	Macro F1	Train (s)	Predict (s)
CNN	**0.9876**	**0.9877**	**0.9875**	**0.9876**	51.6119	0.4285
MLP	0.9826	0.9825	0.9824	0.9825	33.5715	**0.0216**
AdaBoost (depth=3)	0.9175	0.9181	0.9170	0.9170	11019.2396	2.6681
LogisticRegression-NP	0.8684	0.8687	0.8658	0.8655	10.7028	0.0288
AdaBoost (LogReg)	0.8506	0.8511	0.8479	0.8485	48.3919	0.3013
MultinomialNB-NP	0.8356	0.8424	0.8325	0.8341	**0.0541**	0.2689
AdaBoost (stump)	0.7853	0.7880	0.7826	0.7837	241.9720	2.4197
AdaBoost (GaussianNB)	0.5641	0.7158	0.5563	0.5083	46.7646	7.8411



Nhận xét:

CNN dẫn đầu với Accuracy và Macro-F1 cùng đạt 98,77%; MLP theo sát với lần lượt 98,26% và 98,25%. CNN chỉ hơn MLP 0,50 điểm phần trăm.

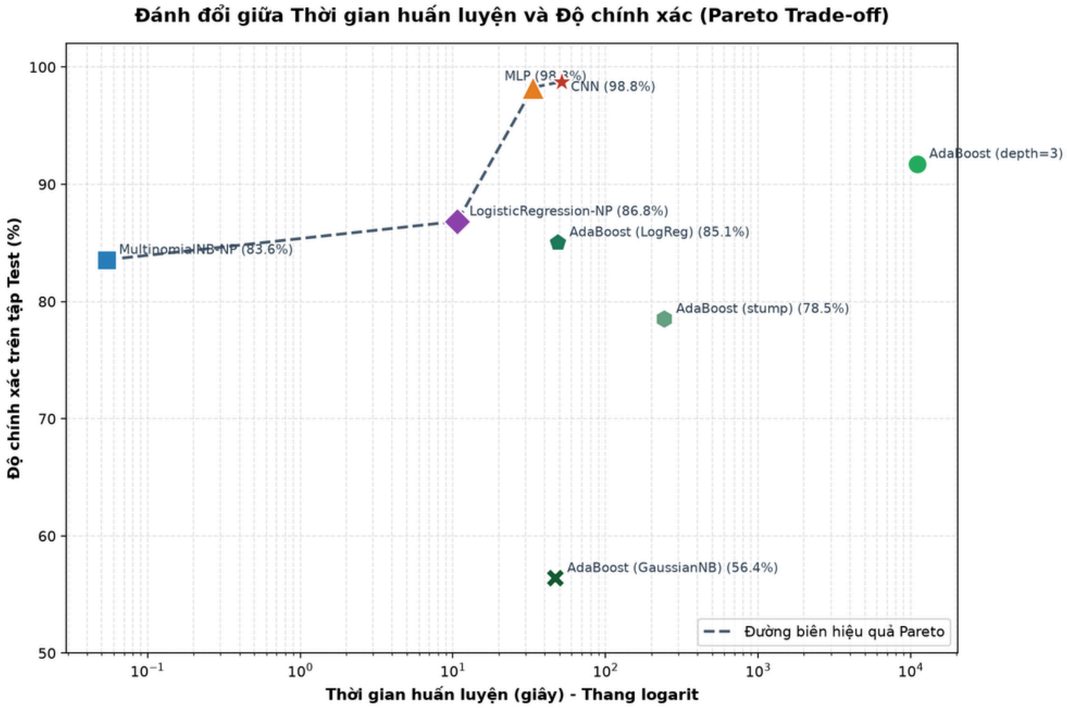
Trong nhóm mô hình truyền thống, AdaBoost (depth=3) đạt kết quả cao nhất (91,75% Accuracy; 91,70% Macro-F1), tiếp theo là LogisticRegression (86.84%; 86,87%).

Các mô hình còn lại có kết quả thấp hơn: AdaBoost (LogReg) đạt 85,06%, MultinomialNB 83,56%, AdaBoost (stump) 78,53% và AdaBoost (GaussianNB) 56,41%.

Thời gian huấn luyện

Bảng tổng hợp metrics và thời gian:

Mô hình	Accuracy	Precision	Recall	Macro F1	Train (s)	Predict (s)
CNN	**0.9876**	**0.9877**	**0.9875**	**0.9876**	51.6119	0.4285
MLP	0.9826	0.9825	0.9824	0.9825	33.5715	**0.0216**
AdaBoost (depth=3)	0.9175	0.9181	0.9170	0.9170	11019.2396	2.6681
LogisticRegression-NP	0.8684	0.8687	0.8658	0.8655	10.7028	0.0288
AdaBoost (LogReg)	0.8506	0.8511	0.8479	0.8485	48.3919	0.3013
MultinomialNB-NP	0.8356	0.8424	0.8325	0.8341	**0.0541**	0.2689
AdaBoost (stump)	0.7853	0.7880	0.7826	0.7837	241.9720	2.4197
AdaBoost (GaussianNB)	0.5641	0.7158	0.5563	0.5083	46.7646	7.8411



Nhận xét:

MultinomialNB nhanh nhất, chỉ mất 0,06 giây.

MLP và CNN lần lượt mất 30,27 giây và 43,59 giây, trong khi LogisticRegression mất 32,29 giây. AdaBoost (depth=3) tốn nhiều thời gian nhất: 11.019,24 giây (khoảng 3,06 giờ), nhưng Accuracy vẫn thấp hơn CNN 7,01 điểm phần trăm.

AdaBoost huấn luyện tuần tự nhiều cây trên pixel đã làm phẳng nên tốn thời gian, nhưng không khai thác được cấu trúc không gian của ảnh như CNN; vì vậy chạy lâu mà Accuracy vẫn thấp hơn.

Thời gian huấn luyện

Bảng hiển thị độ trễ dự đoán mỗi mẫu:

model	độ trễ mẫu (µs/mẫu)
MLP	2.16
LogisticRegression	2.88
MultinomialNB	26.89
AdaBoost (LogReg)	30.13
CNN	42.85
AdaBoost (stump)	241.97
AdaBoost (depth=3)	266.81
AdaBoost (GaussianNB)	784.11

Nhận xét:

MLP và **LogisticRegression** nhanh nhất, lần lượt đạt 2,16 µs/mẫu và 2,88 µs/mẫu.

MultinomialNB mất 26,9 µs/mẫu; CNN mất 42,85 µs/mẫu, vẫn phù hợp với các ứng dụng cần phản hồi nhanh.

Các mô hình boosting có độ trễ cao hơn: AdaBoost (depth=3) mất 266,8 µs/mẫu, còn AdaBoost (GaussianNB) mất 784,1 µs/mẫu.

Outline

1 Tổng quan bài toán
- 1.1. Giới thiệu
- 1.2. Pipeline

2 Dữ liệu & Tiền xử lý (EDA & Preprocessing)

3 Multinomial Naïve Bayes
- 3.1. Lý thuyết MAP
- 3.2. Cài đặt & Thực nghiệm

4 Logistic Regression (OvR)
- 4.1. Lý thuyết
- 4.2. Cài đặt và thực nghiệm

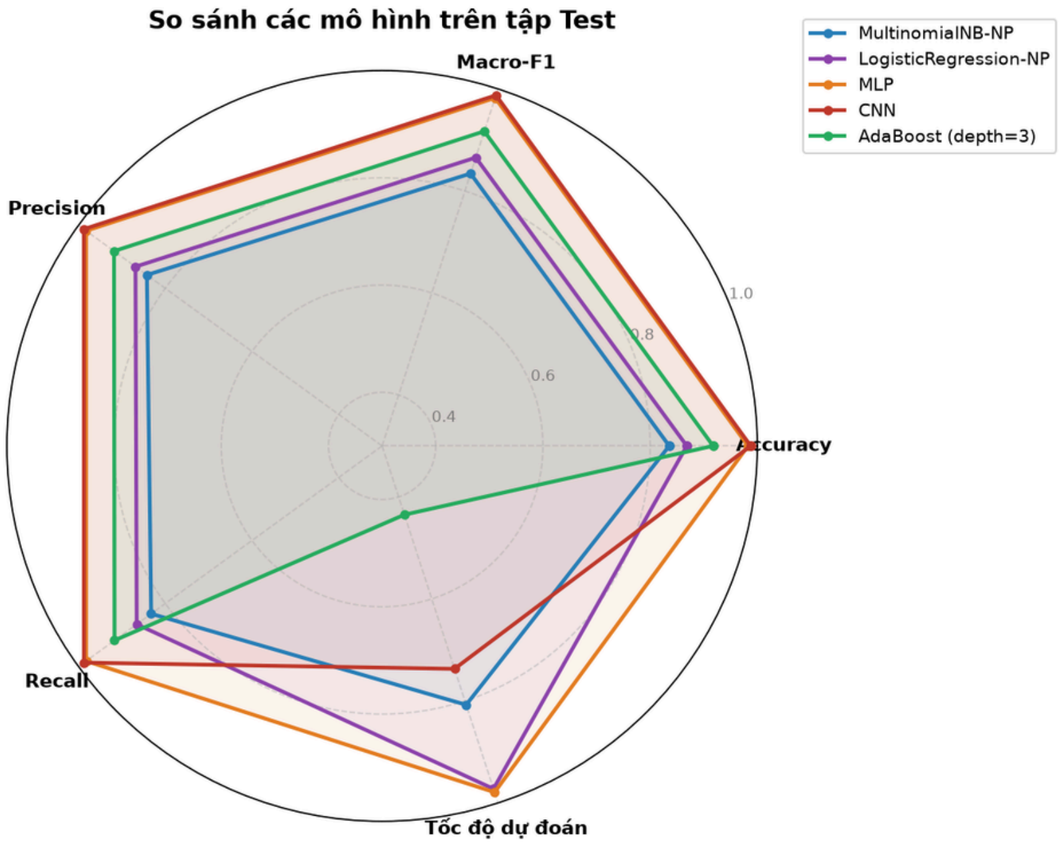
5 MLP & CNN
- 5.1. Lý thuyết, Cài đặt thực nghiệm MLP
- 5.2. Lý thuyết, Cài đặt thực nghiệm CNN
- 5.3. Đối sánh đối chiếu MLP và CNN

6 Ensemble: AdaBoost
- 6.1. Lý thuyết MAP
- 6.2. Cài đặt & Thực nghiệm

7 So sánh & Đánh giá
- 7.1. Khả năng phân loại
- 7.2. Thời gian huấn luyện
- 7.3. Độ trễ từng mẫu

8 Kết luận
- 8.1. Tổng kết
- 8.2. Hướng phát triển

Tổng kết



Mô hình	Accuracy	Precision	Recall	Macro F1	Train (s)	Predict (s)
CNN	**0.9876**	**0.9877**	**0.9875**	**0.9876**	51.6119	0.4285
MLP	0.9826	0.9825	0.9824	0.9825	33.5715	**0.0216**
AdaBoost (depth=3)	0.9175	0.9181	0.9170	0.9170	11019.2396	2.6681
LogisticRegression-NP	0.8684	0.8687	0.8658	0.8655	10.7028	0.0288
AdaBoost (LogReg)	0.8506	0.8511	0.8479	0.8485	48.3919	0.3013
MultinomialNB-NP	0.8356	0.8424	0.8325	0.8341	**0.0541**	0.2689
AdaBoost (stump)	0.7853	0.7880	0.7826	0.7837	241.9720	2.4197
AdaBoost (GaussianNB)	0.5641	0.7158	0.5563	0.5083	46.7646	7.8411

Không có mô hình duy nhất vượt trội trên mọi phương diện:

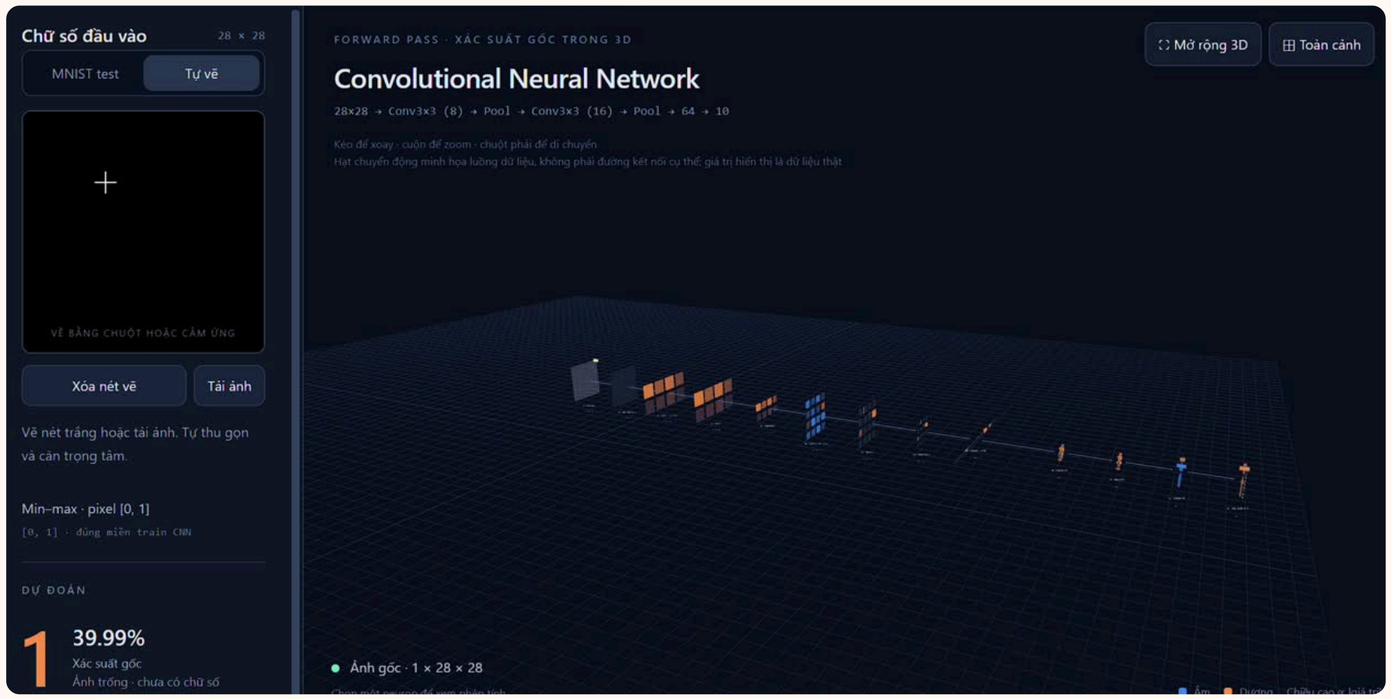
- CNN dẫn đầu về độ chính xác (98,76% Accuracy, độ trễ 42,85 µs/mẫu) nhờ khai thác tốt cấu trúc ảnh 2D.
- MLP tối ưu và cân bằng nhất khi vừa suy diễn nhanh nhất (2,16 µs/mẫu), huấn luyện gọn (33,57 giây) vừa đạt Accuracy 98,26% (chỉ kém CNN 0,50 điểm phần trăm).
- MultinomialNB (huấn luyện 0,054 giây) và LogisticRegression (2,88 µs/mẫu) phù hợp làm baseline với ít khối lượng tính toán.
- AdaBoost (depth=3) kém khả thi do tốn 3,06 giờ huấn luyện và độ trễ cao (266,81 µs/mẫu)

Hướng phát triển

Ứng dụng triển khai trên source đồ án: https://pattern-recognition-mnist-handwritten-digit-recognition.r2u-hcmus.workers.dev/

Web app nhận dạng chữ số viết tay nhằm minh họa trực quan khả năng suy luận của các mô hình được huấn luyện trong đồ án. Website cung cấp các chức năng chính:

- **Nhập liệu:** Vẽ trực tiếp số muốn thử lên canvas, tải ảnh lên hoặc chọn mẫu MNIST có sẵn.
- **Dự đoán và so sánh:** Đối chiếu nhãn dự đoán, độ tin cậy và xác suất 10 lớp của cả 5 mô hình trên cùng một ảnh.
- **Trực quan hóa kiến trúc:** Theo dõi luồng forward pass 3D, quan sát từng lớp, neuron, kernel CNN và feature map.



Các hướng phát triển đồ án khác mà nhóm đề xuất:

Áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) như xoay, dịch chuyển hoặc thêm nhiễu nhằm cải thiện khả năng tổng quát hóa của mô hình. Mở rộng thử nghiệm trên các bộ dữ liệu cùng dạng như chữ số và ký tự viết tay tiếng Việt.

Tài liệu tham khảo & Github

Tài liệu tham khảo:

[1] Trương Phước Hưng, Slide Bài giảng Nhận dạng mẫu: Buổi 1-2 (Bayes Decision Theory), Buổi 3 (Linear Classifiers), Buổi 4 (MLP), Buổi 5(Ensembke) Khoa CNTT, ĐH KHTN ĐHQG-HCM, 2026.

[2] Lê Phú Trường, Source code mẫu: Lab2, Lab3, Lab4, Lab5

[3] Christopher M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006. Mục 4.3.2 (Logistic Regression).

[4] Sergios Theodoridis, Konstantinos Koutroumbas, Pattern Recognition, 4th Edition, Academic Press, 2008.

[5] Yann LeCun, Corinna Cortes, Christopher J.C. Burges, The MNIST Database of Handwritten Digits, 1998.

Github: https://github.com/gautrangtoidayyy/Pattern-Recognition

File pdf báo cáo:

Thanks for your listening!

NHẬN DẠNG CHỮ SỐ VIẾT TAY MNIST

End.